

Berikut adalah penjelasan mendalam dan per baris mengenai kode C# HistogramManager.cs yang Anda berikan.

Secara garis besar, kelas ini berfungsi sebagai **Manager** atau **Facade**. Tugas utamanya adalah menerima gambar (Bitmap) dari aplikasi utama (misalnya dari tombol di UI), memilih algoritma histogram mana yang akan dijalankan (Adaptive, Global, atau Linear), lalu mengembalikan hasilnya.

Berikut adalah detail per bagiannya:

1. Library dan Namespace

C#

```
using System.Drawing;
```

```
using MiniPhotoshop.Logic.Histogram; // PENTING: Panggil folder Logic/Histogram
```

- `using System.Drawing;`: Baris ini mengimpor pustaka standar Windows untuk grafik. Kita membutuhkannya agar kode mengenali tipe data **Bitmap** (objek yang menampung gambar pixel-per-pixel).
- `using MiniPhotoshop.Logic.Histogram;`: Ini mengimpor namespace tempat logika algoritma (matematika) histogram Anda disimpan. Tanpa baris ini, kode tidak akan mengenali kelas seperti `AdaptiveHistogram` atau `GlobalHistogram`.

C#

```
namespace MiniPhotoshop.Logic.Helpers
```

```
{
```

```
    public class HistogramManager
```

```
    {
```

- `namespace MiniPhotoshop.Logic.Helpers`: Ini adalah alamat atau "bungkus" organisasi kode ini. Kelas ini dikategorikan sebagai "Helper" (pembantu).
- `public class HistogramManager`: Definisi kelas utama. Dibuat `public` agar bisa diakses dari Form/Window utama aplikasi Anda.

2. Constructor

C#

```
// Constructor
public HistogramManager()
{
}
```

- `public HistogramManager()`: Ini adalah **Constructor**. Method khusus yang dipanggil saat Anda membuat *instance* baru dari kelas ini (misalnya: `new HistogramManager()`).
- Saat ini isinya kosong, yang berarti tidak ada persiapan khusus yang perlu dilakukan saat kelas ini pertama kali dibuat.

3. Tugas 1: Adaptive Equalization (Metode Utama Anda)

Bagian ini adalah implementasi **CLAHE** (Contrast Limited Adaptive Histogram Equalization) atau varian serupa.

C#

```
// -----
// TUGAS 1: ADAPTIVE EQUALIZATION (Milik Anda)
// -----
public Bitmap ProcessAdaptiveEqualization(Bitmap source)
{
```

- `public Bitmap ProcessAdaptiveEqualization(Bitmap source)`:
 - `public`: Method ini bisa dipanggil dari luar.
 - `Bitmap` (pertama): Tipe data yang akan dikembalikan (output) adalah sebuah gambar baru.
 - `(Bitmap source)`: Inputnya adalah gambar asli yang ingin diedit.

C#

```
if (source == null) return null;
```

- **Pengecekan Keamanan (Null Check):** Sebelum memproses, kode memastikan gambar input (source) benar-benar ada. Jika source kosong (null), method langsung berhenti dan mengembalikan null agar aplikasi tidak *crash*.

C#

```
// Kita pakai grid 8x8 (Standar)  
AdaptiveHistogram algo = new AdaptiveHistogram(8, 8);
```

- AdaptiveHistogram algo: Membuat variabel bernama algo yang bertipe AdaptiveHistogram. Kelas AdaptiveHistogram ini diasumsikan berada di dalam namespace MiniPhotoshop.Logic.Histogram yang diimpor tadi.
- new AdaptiveHistogram(8, 8): Menciptakan objek algoritma tersebut. Angka (8, 8) adalah parameter **Grid Size** (Ukuran Ubin/Tile).
 - **Penjelasan Konsep:** Pada Adaptive Equalization, gambar tidak diproses sekaligus satu layar penuh. Gambar dipotong-potong menjadi kotak-kotak kecil (misalnya 8 baris x 8 kolom). Histogram dihitung di setiap kotak kecil tersebut agar kontras lokal lebih tajam.

C#

```
return algo.Apply(source);  
}
```

- algo.Apply(source): Memerintahkan objek algo untuk menjalankan rumus matematikanya pada gambar source.
- return: Mengirim hasil gambar yang sudah diproses kembali ke pemanggil (UI).

4. Metode Tambahan (Global & Linear)

Bagian ini biasanya dikerjakan oleh rekan tim atau fitur tambahan, tetapi polanya persis sama

dengan metode di atas.

C#

```
// Nanti method teman-teman Anda ditaruh di bawah sini...
```

```
// Global Equalization
```

```
public Bitmap ProcessGlobalEqualization(Bitmap source)
{
    if (source == null) return null;
    GlobalHistogram algo = new GlobalHistogram();
    return algo.Apply(source);
}
```

- **Perbedaan:** Di sini objek yang dibuat adalah new GlobalHistogram().
- **Konsep:** Global Equalization menghitung histogram dari **seluruh gambar sekaligus** (bukan dipotong-potong). Ini lebih cepat tapi kadang membuat area yang sudah terang menjadi terlalu silau (overexposed).

C#

```
public Bitmap ProcessLinearStretch(Bitmap source)
{
    if (source == null) return null;
    LinearStretch algo = new LinearStretch();
    return algo.Apply(source);
}
}
```

- **Perbedaan:** Objek yang dibuat adalah new LinearStretch().
 - **Konsep:** Linear Stretch (Contrast Stretching) merenggangkan nilai pixel tergelap menjadi 0 dan paling terang menjadi 255 secara linear. Ini berbeda dengan equalization yang menggunakan probabilitas (CDF).
-

Rangkuman Alur Kerja

1. **UI** memanggil HistogramManager.
2. **UI** mengirim gambar (Bitmap) ke method ProcessAdaptiveEqualization.
3. **HistogramManager** membuat "pekerja" spesialis bernama AdaptiveHistogram dengan setelan grid 8x8.
4. **AdaptiveHistogram** melakukan perhitungan matematika berat di belakang layar (melalui method Apply).
5. Hasil gambar yang sudah tajam dikembalikan ke **UI** untuk ditampilkan.

Apakah Anda ingin saya menjelaskan secara spesifik bagaimana logika matematika di dalam kelas AdaptiveHistogram (bagian tiling 8x8 tersebut) biasanya bekerja?